Module 10 Structured Note: String Permutations with Recursion and Backtracking

1. What this module is about

Module 10 teaches how to generate **permutations of a string**.

A permutation is a different arrangement of the same characters.

Example:

```
ABC
```

Permutations of `ABC` are:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

So the main skill in this module is:

```
Choose a character.
Move to the next position.
Undo the choice.
Try another choice.
```

This is the basic idea of **recursion and backtracking**.

---

2. Required ideas from previous modules

Before learning permutations, remember these ideas from earlier modules.

C-style strings end with `\0`

A C-style string is a character array ending with the null terminator:

```
'\0'
```

Example:

```
char s[] = "ABC";
```

Memory view:

```
Index:  0   1   2   3
Value:  A   B   C   \0
```

The visible string is:

```
ABC
```

The `\0` is not part of the visible word. It only tells C where the string ends.

---

3. What is a permutation?

A **permutation** is an arrangement of characters.

If the string is:

```
ABC
```

Then one possible arrangement is:

```
ABC
```

Another possible arrangement is:

```
BAC
```

Another one is:

```
CBA
```

All of these use the same characters:

```
A, B, C
```

But the order is different.

That is why they are called permutations.

---

4. Important rule of permutations

Each permutation must use every character exactly once.

For `ABC`, each final answer must contain:

```
one A
one B
one C
```

Correct examples:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

Incorrect examples:

```
AAA
ABB
CCB
AB
ABCD
```

Why are these incorrect?

Example	Problem
`AAA`	Uses `A` too many times
`ABB`	Uses `B` too many times and misses `C`
`CCB`	Uses `C` too many times and misses `A`
`AB`	Missing `C`
`ABCD`	Has an extra character `D`

A valid permutation must rearrange the original characters only.

---

5. Factorial: How many permutations are possible?

If a string has length `n`, the number of permutations is:

```
n!
```

This is read as:

```
n factorial
```

Factorial means multiplying from `n` down to `1`.

Examples:

```
1! = 1
2! = 2 × 1 = 2
3! = 3 × 2 × 1 = 6
4! = 4 × 3 × 2 × 1 = 24
5! = 5 × 4 × 3 × 2 × 1 = 120
```

So:

String length	Number of permutations
1	1
2	2
3	6

String length	Number of permutations
4	24
5	120
10	3,628,800

This grows very quickly.

That is why permutation algorithms can become expensive.

---

6. Why `ABC` has 6 permutations

For:

```
ABC
```

There are 3 characters.

For the first position, we have 3 choices:

```
A _ _
B _ _
C _ _
```

After choosing the first character, only 2 characters are left.

For the second position, we have 2 choices.

After choosing the second character, only 1 character is left.

So the total number of arrangements is:

```
3 × 2 × 1 = 6
```

Therefore:

```
3! = 6
```

---

7. State space tree

A **state space tree** is a tree of choices.

It shows all possible decisions the algorithm can make.

For `ABC`, the first choice is the first character:

```
        start
      /   |
     A    B    C
```

If we choose `A`, the remaining characters are `B` and `C`:

```
        start
          |
          A
         /
        B     C
```

If we choose `A` then `B`, only `C` remains:

```
        start
          |
          A
         /
        B     C
        |     |
        C     B
```

This gives:

```
ABC
ACB
```

The complete strings at the bottom of the tree are called **leaf nodes**.

Each leaf node is one full permutation.

---

8. Brute force idea

Generating permutations is a type of **brute force** method.

Brute force means:

```
Try all possible choices.
Print every valid result.
```

For permutations, this means we generate every possible arrangement.

This is useful when we really need all arrangements, but it can be slow for long strings because the number of outputs grows as `n!`.

---

9. What is recursion?

Recursion means a function calls itself.

In this module, recursion is used to fill one position of the result at a time.

Simple idea:

```
Fill position 0.
Fill position 1.
Fill position 2.
Print the result.
```

For `ABC`:

```
position 0 = A
position 1 = B
position 2 = C
```

Now the result is:

```
ABC
```

Then recursion goes back and tries another choice.

---

10. What is backtracking?

Backtracking means undoing a choice after trying it.

The pattern is:

```
Choose something.
Use recursion to continue.
Undo the choice.
Try another choice.
```

Example:

```
Choose A
Choose B
Choose C
Print ABC
Undo C
Undo B
Choose C
Choose B
Print ACB
```

Backtracking is necessary because after one branch is finished, we need to restore the previous state and explore another branch.

---

11. Main mental model

The whole module can be remembered like this:

```
At each position:
    Try every possible unused character.
    Recurse to fill the next position.
    Undo the choice.
```

This is the core logic behind both permutation methods.

Part A: Method 1 — Flag array and result array

12. Main idea of Method 1

In Method 1, we use two helper arrays:

```
flag array
result array
```

The **flag array** remembers which characters are already used.

The **result array** stores the current permutation being built.

Example input:

```
char s[] = "ABC";
```

Indexes:

```
s[0] = A
s[1] = B
s[2] = C
```

The result array starts empty:

```
res = _ _ _
```

The flag array starts with all zeros:

```
flag = 0 0 0
```

Meaning:

```
0 means unused
1 means already used
```

---

13. Meaning of `flag[i]`

The flag array tells whether a character has already been used in the current permutation.

For:

```
ABC
```

The flags are:

```
flag[0] -> A
flag[1] -> B
flag[2] -> C
```

If:

```
flag = 1 0 0
```

That means:

```
A is already used
B is not used
C is not used
```

If:

```
flag = 1 1 0
```

That means:

```
A is already used
B is already used
C is not used
```

---

14. Meaning of `res[k]`

The result array stores the current permutation.

The variable `k` tells which position we are filling.

Example:

```
res = A _ _
k = 1
```

This means position `0` is already filled with `A`, and now we are filling position `1`.

If we choose `B` next:

```
res = A B _
k = 2
```

Now the next position to fill is position `2`.

---

15. Base case in Method 1

The base case is the stopping condition for recursion.

In Method 1, the base case happens when `k` reaches the end of the string.

Code idea:

```
if (s[k] == '\0') {
    res[k] = '\0';
    printf("%s\n", res);
}
```

Meaning:

```
If k is at the end of the string,
then the result array is full,
so print the permutation.
```

For `ABC`:

```
s[0] = A
s[1] = B
```

```
s[2] = C
s[3] = \0
```

When `k == 3`, the permutation is complete.

---

16. Method 1 code

```c
#include <stdio.h>

void perm(char s[], int k) {
    static int flag[10] = {0};
    static char res[10];
    int i;

    if (s[k] == '\0') {
        res[k] = '\0';
        printf("%s\n", res);
    } else {
        for (i = 0; s[i] != '\0'; i++) {
            if (flag[i] == 0) {
                res[k] = s[i];
                flag[i] = 1;

                perm(s, k + 1);

                flag[i] = 0;
            }
        }
    }
}

int main() {
    char s[] = "ABC";
    perm(s, 0);
    return 0;
}
```

Output:

```
ABC
ACB
BAC
BCA
```

```
CAB
CBA
```

---

17. Understanding the static arrays

The code uses:

```
static int flag[10] = {0};
static char res[10];
```

`static` means these arrays keep their values between recursive calls.

This is useful because every recursive call needs to see the same `flag` and `res` arrays.

In beginner words:

```
flag remembers which characters are already used.
res remembers the permutation being built.
```

---

18. Understanding the loop in Method 1

The loop is:

```
for (i = 0; s[i] != '\0'; i++) {
```

This means:

```
Try every character in the input string.
```

Inside the loop:

```
if (flag[i] == 0) {
```

This means:

```
Use this character only if it has not already been used.
```

Then:

```
res[k] = s[i];
flag[i] = 1;
```

Meaning:

```
Put the character into the current result position.
Mark the character as used.
```

Then:

```
perm(s, k + 1);
```

Meaning:

```
Use recursion to fill the next position.
```

Finally:

```
flag[i] = 0;
```

Meaning:

```
Undo the choice.
Make this character available again.
```

That final line is backtracking.

---

19. Dry run of Method 1 for `ABC`

Input:

```
ABC
```

Start:

```
k = 0
res = _ _ _
flag = 0 0 0
```

Step 1: Choose `A`

At `k = 0`, choose `A`.

```
res[0] = A
flag[0] = 1
```

Now:

```
res = A _ _
flag = 1 0 0
```

Call:

```
perm(s, 1);
```

Step 2: Choose `B`

At `k = 1`, scan from the beginning.

`A` is skipped because it is already used.

Choose `B`.

```
res[1] = B
flag[1] = 1
```

Now:

```
res = A B _
flag = 1 1 0
```

Call:

```
perm(s, 2);
```

Step 3: Choose `C`

At `k = 2`, `A` and `B` are already used.

Choose `C`.

```
res[2] = C
flag[2] = 1
```

Now:

```
res = A B C
flag = 1 1 1
```

Call:

```
perm(s, 3);
```

Step 4: Print `ABC`

Now:

```
s[3] == '\0'
```

So the result is complete.

Print:

```
ABC
```

Step 5: Backtrack from `C`

After printing, the function returns.

Now undo the last choice:

```
flag[2] = 0;
```

Now:

```
C is available again.
```

The program then goes back and tries other choices.

Step 6: Backtrack from `B` and choose `C`

After finishing the `ABC` branch, undo `B`:

```
flag[1] = 0
```

Now choose `C` for position `1`:

```
res = A C _
flag = 1 0 1
```

Then choose `B` for position `2`:

```
res = A C B
```

Print:

```
ACB
```

So the first two permutations are:

```
ABC
ACB
```

The same process continues for branches starting with `B` and `C`.

Final result:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

---

20. Method 1 summary

Method 1 works like this:

```
Use flag[] to remember used characters.
Use res[] to build the answer.
Use k to track the current result position.
When the result is full, print it.
Backtrack by resetting flag[i] to 0.
```

Pattern:

```
choose character
put it into res[k]
mark it as used
recurse
unmark it
```

Part B: Method 2 — In-place swapping

21. Main idea of Method 2

Method 2 generates permutations by swapping characters inside the same string.

Instead of using a separate result array, it rearranges the original array.

Example:

```
ABC
```

At position `0`, we try placing every character there:

```
ABC   A is at position 0
BAC   B is at position 0
CBA   C is at position 0
```

Then recursion handles the remaining positions.

---

22. Important variables in Method 2

The method uses two indexes:

```
l = current position / left side
h = last index / high side
```

For:

```
char s[] = "ABC";
```

Indexes are:

```
Index: 0 1 2
Value: A B C
```

So we call:

```
permSwap(s, 0, 2);
```

Meaning:

```
Start arranging from index 0.
Last valid character is at index 2.
```

---

23. Base case in Method 2

The base case is:

```
if (l == h) {
    printf("%s\n", s);
}
```

Meaning:

```
If the current position reaches the last index,
the string is fully arranged,
so print it.
```

For `ABC`, when `l == 2`, the last character is fixed.

So the permutation is complete.

---

24. Swap function

Method 2 needs a helper function to swap two characters.

```
void swap(char *x, char *y) {
    char temp = *x;
    *x = *y;
    *y = temp;
}
```

This swaps the values at two memory locations.

Example:

```
x points to A
y points to B
```

After swapping:

```
x has B
y has A
```

The temporary variable `temp` is needed so we do not lose one value during the swap.

---

25. Method 2 code

```c
#include <stdio.h>

void swap(char *x, char *y) {
    char temp = *x;
    *x = *y;
    *y = temp;
}

void permSwap(char s[], int l, int h) {
    int i;

    if (l == h) {
        printf("%s\n", s);
    } else {
        for (i = l; i <= h; i++) {
            swap(&s[l], &s[i]);

            permSwap(s, l + 1, h);

            swap(&s[l], &s[i]);
        }
    }
}

int main() {
    char s[] = "ABC";
    permSwap(s, 0, 2);
    return 0;
}
```

Possible output:

```
ABC
ACB
```

```
BAC
BCA
CBA
CAB
```

The order may differ depending on the implementation, but the set of permutations should be correct.

---

26. Understanding the loop in Method 2

The loop is:

```
for (i = l; i <= h; i++) {
```

This means:

```
Try every character from the current position to the end.
```

Inside the loop:

```
swap(&s[l], &s[i]);
```

This means:

```
Put one possible character into the current position.
```

Then:

```
permSwap(s, l + 1, h);
```

This means:

```
Recursively arrange the remaining part of the string.
```

Finally:

```
swap(&s[l], &s[i]);
```

This means:

```
Undo the swap and restore the string.
```

That final swap is backtracking.

---

27. Dry run of Method 2 for `ABC`

Start:

```
s = ABC
l = 0
h = 2
```

At `l = 0`, we try each character at position `0`.

Branch 1: Keep `A` at position 0

Swap:

```
swap s[0] with s[0]
```

The string remains:

```
ABC
```

Now recurse:

```
permSwap(s, 1, 2)
```

Now position `0` is fixed as `A`, and we arrange the remaining part:

```
B C
```

This produces:

```
ABC
ACB
```

Then the program swaps back to restore the previous state.

Branch 2: Put `B` at position 0

Swap:

```
swap s[0] with s[1]
```

The string becomes:

```
BAC
```

Now recurse:

```
permSwap(s, 1, 2)
```

Position `0` is fixed as `B`, and we arrange the remaining part:

```
A C
```

This produces:

```
BAC
BCA
```

Then the program swaps back to restore the previous state.

Branch 3: Put `C` at position 0

Swap:

```
swap s[0] with s[2]
```

The string becomes:

```
CBA
```

Now recurse:

```
permSwap(s, 1, 2)
```

Position `0` is fixed as `C`, and we arrange the remaining part:

```
B A
```

This produces:

```
CBA
CAB
```

Final output:

```
ABC
ACB
BAC
BCA
CBA
CAB
```

---

28. Why the second swap is necessary

This line is very important:

```
swap(&s[l], &s[i]);
```

when it appears after recursion.

Example:

```
ABC
```

If we swap `A` and `B`, the string becomes:

```
BAC
```

After finishing the branch starting with `B`, we must restore it back to:

```
ABC
```

If we do not restore it, the next branch starts from the wrong string.

So the correct pattern is:

```
swap before recursion
recurse
swap back after recursion
```

This is backtracking in the swapping method.

---

29. Method 2 summary

Method 2 works like this:

```
Use l to choose the current position.
Try each character from l to h in that position.
Swap the character into position l.
Recurse for l + 1.
Swap back to restore the string.
```

Pattern:

```
swap choice into current position
recurse
swap back
```

Part C: Comparing the two methods

30. Method comparison table

Feature	Flag/result array method	In-place swapping method
Main idea	Build a separate result	Rearrange the original string
Extra result array	Yes	No
Uses flag array	Yes	No
Uses swapping	No	Yes
Beginner friendly	Easier to trace	Slightly harder
Backtracking step	Reset `flag[i] = 0`	Swap back
Original string changed?	No	Temporarily yes, but restored

---

31. Which method should a beginner learn first?

A beginner should usually learn the **flag/result array method** first.

Why?

Because it clearly shows:

```
which characters are used
which result is being built
which position is being filled
```

After that, learn the **in-place swapping method**.

The swapping method is shorter and uses less helper storage, but it is easier to make mistakes if you forget to swap back.

Part D: Complexity analysis

32. Time complexity

For a string of length `n`, the number of permutations is:

```
n!
```

Each permutation has length `n`.

If we print every permutation, printing one permutation takes:

```
O(n)
```

Since there are `n!` permutations, the total time is:

```
O(n × n!)
```

This is very large.

Example:

```
3!  = 6
4!  = 24
5!  = 120
10! = 3,628,800
```

So permutations become expensive quickly.

---

33. Space complexity

The recursion depth is `n` because the function fills one position at a time.

So the recursion stack uses:

```
O(n)
```

Method 1 also uses:

```
flag array
result array
```

Both are size `n`, so Method 1 also uses:

```
O(n)
```

Method 2 does not use a result array, but it still uses recursive calls.

So Method 2 also has recursion stack space:

```
O(n)
```

---

34. Complexity table

Method	Time complexity	Extra space
Flag/result array method	`O(n × n!)`	`O(n)`
In-place swapping method	`O(n × n!)`	`O(n)` recursion stack

Important note:

Even if the code uses little extra memory, the output itself is huge because there are `n!` permutations.

Part E: Common beginner mistakes

35. Mistake 1: Forgetting the base case

Wrong idea:

```
perm(s, k + 1);
```

without a clear stopping condition.

Problem:

```
The recursion may continue incorrectly.
```

Correct idea:

```c
if (s[k] == '\0') {
    res[k] = '\0';
    printf("%s\n", res);
}
```

The base case tells recursion when to stop.

---

36. Mistake 2: Not backtracking the flag

Wrong:

```c
flag[i] = 1;
perm(s, k + 1);
```

Correct:

```c
flag[i] = 1;
perm(s, k + 1);
flag[i] = 0;
```

Why?

Because after one branch finishes, that character must become available again for another branch.

---

37. Mistake 3: Not swapping back

Wrong:

```c
swap(&s[l], &s[i]);
permSwap(s, l + 1, h);
```

Correct:

```c
swap(&s[l], &s[i]);
permSwap(s, l + 1, h);
swap(&s[l], &s[i]);
```

The second swap restores the string for the next branch.

38. Mistake 4: Thinking permutations are cheap

Permutations are not cheap.

The number of outputs grows factorially.

For example:

```
10! = 3,628,800
```

So do not start testing with a long string.

Start with:

```
AB
ABC
ABCD
```

---

39. Mistake 5: Confusing combinations and permutations

Permutations care about order.

Example:

```
ABC
BAC
```

These are different permutations because the order changed.

Combinations do not care about order.

But this module is about **permutations**, so order matters.

---

40. Mistake 6: Forgetting `\0` in the result array

In Method 1, before printing `res`, write:

```
res[k] = '\0';
```

Why?

Because `printf("%s", res)` prints until it finds `\0`.

Without `\0`, C may keep reading memory beyond the result array.

---

41. Mistake 7: Using a string literal for in-place swapping

Avoid this when modifying the string:

```
char *s = "ABC";
```

Better:

```
char s[] = "ABC";
```

Why?

Because `char s[] = "ABC";` creates a modifiable character array.

The swapping method changes the string temporarily, so the string must be mutable.

Part F: Main code patterns to remember

42. Pattern 1: Flag/result array method

```
void perm(char s[], int k) {
    static int flag[10] = {0};
    static char res[10];
    int i;

    if (s[k] == '\0') {
        res[k] = '\0';
        printf("%s\n", res);
    } else {
        for (i = 0; s[i] != '\0'; i++) {
            if (flag[i] == 0) {
```

```
                res[k] = s[i];
                flag[i] = 1;
                perm(s, k + 1);
                flag[i] = 0;
            }
        }
    }
}
```

Remember:

```
flag[i] = 1 means choose.
flag[i] = 0 means undo.
```

---

43. Pattern 2: Swap function

```c
void swap(char *x, char *y) {
    char temp = *x;
    *x = *y;
    *y = temp;
}
```

This safely swaps two characters.

---

44. Pattern 3: In-place swapping method

```c
void permSwap(char s[], int l, int h) {
    int i;

    if (l == h) {
        printf("%s\n", s);
    } else {
        for (i = l; i <= h; i++) {
            swap(&s[l], &s[i]);
            permSwap(s, l + 1, h);
            swap(&s[l], &s[i]);
        }
    }
}
```

Remember:

```
first swap = choose
recursive call = continue
second swap = undo
```

Part G: Practice tasks

45. Practice 1

Write all permutations of:

```
AB
```

Expected answer:

```
AB
BA
```

---

46. Practice 2

Write all permutations of:

```
ABC
```

Expected answer:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

The order may differ depending on the method, but all six should appear.

---

47. Practice 3

Dry run the flag/result array method for:

```
ABC
```

Track:

```
k
res
flag
```

At each step, ask:

```
Which character is being chosen?
Which flag becomes 1?
When does it become 0 again?
```

---

48. Practice 4

Dry run the swapping method for:

```
ABC
```

Track:

```
s
l
h
i
```

At each step, ask:

```
Which two characters are swapped?
When are they swapped back?
```

---

49. Practice 5

Calculate the number of permutations for:

```
ABCD
```

Since the length is 4:

```
4! = 4 × 3 × 2 × 1 = 24
```

So `ABCD` has 24 permutations.

---

50. Final summary

Module 10 teaches how to generate all permutations of a string.

The key ideas are:

```
A permutation is a different arrangement of the same characters.
A string of length n has n! permutations.
A state space tree shows all possible choices.
Recursion fills one position at a time.
Backtracking undoes a choice so another choice can be tried.
```

There are two main methods:

```
Method 1: Use flag[] and res[].
Method 2: Use in-place swapping.
```

Method 1 pattern:

```
choose character
put it in result
mark as used
recurse
unmark it
```

Method 2 pattern:

```
swap character into position
recurse
swap back
```

Most important final idea:

```
Permutation = choose + recurse + undo
```

If you understand that pattern, you understand the heart of Module 10.